VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



ASSIGNMENT 2 REPORT

DEEP LEARNING AND ITS APPLICATIONS

Course Code: CO3133 Class: CC01

SEMESTER 261

Group Name: Group 09 - Ryzen9

Instructor(s): Le Thanh Sach

Student(s): Hoang Kim Cuong - 2352145

Phan Tan Loc - 2352712

Tran Manh Thang - 2353124

HO CHI MINH CITY, NOVEMBER 2026

Workload

No.	Full name	Student ID	Workload (%)	Assigned Report Section
1	Hoang Kim Cuong	2352145	100%	Implementation Results
2	Phan Tan Loc	2352712	100%	Problem and Data Description
3	Tran Manh Thang	2353124	100%	Methodology

Contents

List of Figures **3**

List of Tables **3**

1 Problem and Data Description **5**
1.1 Problem Formulation . . . 5
1.2 Dataset Description . . . 6
1.2.1 Dataset Proposal and Approval . . . 6
1.2.2 Data Quality Audit . . . 7
1.2.3 Bias and Limitations . . . 8
1.3 Exploratory Data Analysis . . . 8
1.3.1 Class Distribution and Imbalance . . . 8
1.3.2 Input Size . . . 9
1.3.3 Representative Samples and Expected Difficulties . . . 9
1.4 Data Splitting . . . 10
1.5 Preprocessing . . . 11
1.5.1 Transforms . . . 11
1.5.2 Example Batch After Preprocessing . . . 11

2 Methodology **13**
2.1 Pipeline Overview . . . 13
2.2 Baseline and Main Methods . . . 13
2.2.1 Scratch CNN Baseline . . . 14
2.2.2 Pretrained ResNet-50 . . . 15
2.2.3 Fine-tuning Procedure . . . 17
2.2.4 Loss and Metrics . . . 17
2.2.5 Implementation Components . . . 17
2.3 Training Setup . . . 18
2.3.1 Shared Hyperparameters . . . 18
2.3.2 Early Stopping and Checkpoint Selection . . . 18
2.3.3 Hyperparameter Selection . . . 19
2.3.4 Experimental Environment and Training Time . . . 19
2.4 Experimental Design . . . 19
2.4.1 Compute-Cost Measurement . . . 20

3 Implementation Results **21**
3.1 Training Behavior . . . 21
3.1.1 Learning Curves and Selected Checkpoints . . . 21
3.1.2 Underfitting and Overfitting . . . 22
3.1.3 Implementation Issues and Fixes . . . 22
3.2 Quantitative Results . . . 22
3.2.1 Predictive Performance . . . 22
3.2.2 Compute Cost . . . 23
3.3 Qualitative Results . . . 23
3.3.1 Correct and Incorrect Predictions . . . 23
3.4 Comparison and Discussion . . . 24

3.4.1 Hypothesis Assessment . . . 24
3.4.2 Where the Differences Appear . . . 24
3.4.3 Likely Causes . . . 25
3.4.4 Accuracy–Speed–Parameter Trade-offs . . . 26
3.4.5 Conclusions Not Yet Supported . . . 26
3.5 Error Analysis . . . 26
3.5.1 Error Groups . . . 27
3.5.2 Cross-Family Errors: Sample Review . . . 28
3.5.3 Confidence of Errors and Errors Shared by All Models . . . 29
3.6 Ablation Study . . . 30
3.7 Limitations and Conclusion . . . 31
3.7.1 Limitations . . . 31
3.7.2 Conclusion and Future Work . . . 31

4 AI Usage Disclosure **32**
4.1 Project Scaffold, Environment, and Experiment Runner . . . 32
4.2 Data Audit, Splitting, and EDA . . . 32
4.3 Models, Training, and Evaluation Code . . . 33
4.4 Final Results Analysis, Assignment Page, and Report . . . 33
4.5 Results Provenance and Responsibility . . . 34

5 Reproducibility **35**

References **36**

List of Figures

1 Examples of identical images filed under two different dishes; both copies were removed. . . . 8
2 Retained images per class in the training, validation, and test splits. . . . 9
3 Distributions of image width, height, and aspect ratio. . . . 9
4 One representative image from each of the 30 food classes. . . . 10
5 Training images after augmentation, de-normalized for display and titled with their labels. . . . 12
6 The shared pipeline from raw images to evaluation. . . . 13
7 Training and validation curves of the three final runs; dotted lines mark the selected epochs. . . . 21
8 Confident correct, low-confidence correct, and confident wrong test predictions of full ResNet-50 (T = true class, P = predicted class, with softmax confidence). . . . 24
9 Test confusion matrix of full ResNet-50, with true classes in rows and each row normalized by the class support. . . . 27
10 Thirty randomly sampled cross-family errors of full ResNet-50 (seed 0), with true class, prediction, and confidence. . . . 29

List of Tables

1 30VNFoods dataset properties. 6
2 Data quality audit of 30VNFoods version 11. 7
3 The shared group-aware split used by all runs. 10
4 Preprocessing and augmentation applied to each split. 11
5 Architecture, initialization, and parameter counts of the three configurations. 14
6 Layer-by-layer architecture and parameter count of the scratch CNN (output shapes exclude the batch dimension). 15
7 Stage-by-stage architecture and parameter count of ResNet-50 with the 30-class head (output shapes exclude the batch dimension). 16
8 Shared training settings used for all three runs. 18
9 Run identifiers, completed epochs, and training times of the three final runs. 19
10 Hypotheses, factors, metrics, and comparisons of the two experiments. . . . 20
11 Validation and test performance of the selected checkpoints (F1 denotes macro-F1). 22
12 Training and inference cost on one Kaggle Tesla T4 (Frozen and Full denote the two ResNet-50 regimes). 23
13 Macro-F1 gains and verdicts for the two experiments. 24
14 Per-class test F1 of the scratch CNN and the frozen and fully fine-tuned ResNet-50, with test support. 25
15 Test errors of full ResNet-50 grouped by the dish families of the true and predicted classes. 28
16 Test errors of the three models grouped by prediction confidence (Frozen and Full denote the two ResNet-50 regimes). 30
17 Controlled ablation: frozen backbone versus full fine-tuning of ResNet-50. . 30
18 Saved runs underlying the reported final results. 35

1 Problem and Data Description

1.1 Problem Formulation

Food delivery, review, and nutrition-tracking applications need to know which dish appears in a photograph without asking the user to type its name. A recognized dish can tag uploaded photos for search, suggest calories for a logged meal, or retrieve recipes and restaurants from a single picture. General-purpose models do not solve this directly: ImageNet contains no Vietnamese dish classes, many dishes look alike in a bowl, and the same dish varies by region, vendor, and plating. This assignment studies the first building block of such applications: recognizing one of 30 common Vietnamese dishes from a single photograph.

The task follows the *image classification* track of Assignment 2. The prediction unit is one photograph, and each photograph receives exactly one of $K = 30$ mutually exclusive dish labels. Let $x \in \mathbb{R}^{3\times224\times224}$ denote a preprocessed RGB image and $y \in \{0, \dots, 29\}$ its class index. A classifier f_θ maps x to K logits $z = f_\theta(x) \in \mathbb{R}^{30}$, and the predicted class is[1]

$$\hat{y} = \arg\max_k \hat{p}_k, \qquad \hat{p}_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}. \tag{1}$$

Here, $\hat{p}_k$ is the softmax probability of class k, and θ contains the trainable parameters of f_θ. The slides index classes from 1 to K; the dataset stores class indices from 0 to 29.

Training minimizes the mean multiclass cross-entropy over a mini-batch of n images:[2]

$$\mathcal{L}_{\text{CE}} = -\frac{1}{n}\sum_{i=1}^{n}\sum_{k=1}^{K} y_{ik} \log \hat{p}_{ik} = -\frac{1}{n}\sum_{i=1}^{n} \log \hat{p}_{i,y_i}. \tag{2}$$

Here, y_{ik} is the one-hot target and $\hat{p}_{ik}$ the softmax probability of class k for image i. The loss takes raw logits, so no separate softmax layer is used during training.

The assignment compares a simple CNN trained from scratch with an ImageNet-pretrained ResNet-50 and then runs a controlled experiment on how the pretrained backbone is fine-tuned. Two research questions guide the work:

- **Experiment A (baseline versus pretrained):** does an ImageNet-pretrained ResNet-50 outperform a CNN trained from scratch on about 17,500 training images?
- **Experiment B (frozen versus full fine-tuning):** with every other factor fixed, does updating the whole backbone improve validation macro-F1 over a frozen backbone, and at what extra cost?

The main difficulties are:

- **Fine-grained classes.** Several noodle soups share broth, herbs, and bowl shape, and steamed cakes wrapped in banana leaves look alike from the outside.
- **Large intra-class variation.** The same dish changes with region, vendor, portion, plating, camera angle, and lighting.
- **Background clutter.** Photographs often contain side dishes, hands, tables, or several dishes, although each image carries one label.

- **Class imbalance.** Class sizes range from 441 to 1,517 retained images, so accuracy alone would reward the frequent dishes.
- **Web-collected data.** Exact and near duplicates, some filed under two classes, must be removed or grouped before splitting to avoid leakage.
- **Compute budget.** Every run must fit a single GPU session, so training time and inference cost are reported alongside predictive quality.

1.2 Dataset Description

The experiments use 30VNFoods, published on Kaggle by Quan Dang as `quandang/vietnamese-foods`.[14] Kaggle version 11 was downloaded and audited on 03 October 2026. The Kaggle page lists the CC BY-NC-SA 4.0 license, while the downloaded archive also contains an MIT `LICENSE` file. Both records are preserved; the stricter CC BY-NC-SA terms are applied, the data is used for non-commercial coursework only, and the images are not redistributed in the repository.

Images were collected from the web, and the archive lists source URLs per class. Each image has one dish label taken from its folder name. The archive is distributed with an official split of 17,581 training, 2,515 validation, and 5,040 test images (approximately 70/10/20); Section 1.4 explains why this split is replaced. Table 1 summarizes the dataset.

Property	Value
Task	Single-label, 30-class image classification
Source and version	Kaggle `quandang/vietnamese-foods`, version 11
License	CC BY-NC-SA 4.0 (Kaggle page); MIT file in the archive
Original samples	25,136 images, all readable
Retained samples	24,982 images after the duplicate audit
Annotations	One dish name per image, taken from the folder name
Storage	4.48 GB ZIP archive; median file size 127 KB
File formats	All named `.jpg`; content is 24,854 JPEG, 119 PNG, 7 MPO, and 2 BMP files, all opened by content and converted to RGB

Table 1: 30VNFoods dataset properties.

1.2.1 Dataset Proposal and Approval

The dataset proposal was submitted at Milestone M1 and records the dataset name, source, version, and license; the task, input, and output; sample counts and annotation type; a preliminary distribution analysis; the splitting plan and split unit; the evaluation metrics; the baseline plan; candidate pretrained models (EfficientNet-B0, ResNet-50, ConvNeXt-Tiny, ViT-B/16, and MobileNetV3-Large); and a compute estimate. Its approval status is **Approved** by the instructor, as recorded on the assignment page. The dataset meets the image-classification thresholds of the handbook: 30 classes (at least 5 required), 17,486 training images (at least 5,000 required), and at least 309 training and 66 test images per class. The final protocol differs from the proposal in two documented

ways: ResNet-50 was selected among the candidates, and the provisional split counts were replaced by the counts after duplicate review.

1.2.2 Data Quality Audit

The audit opened every file, hashed it, and searched for duplicates before any split was made. Table 2 lists the exclusions. Exact copies were found with SHA-256. Near-duplicate candidates came from a 64-bit difference hash (dHash) with Hamming distance at most 8, which yielded 1,718 candidate pairs. A candidate pair was confirmed only when the mean absolute difference between 32×32 grayscale thumbnails was at most 30/255. Without this confirmation step, similar plate layouts chained into groups of more than 250 images; with it, the largest group contains 9 images.

Audit step	**Images**
Images in the archive (none unreadable)	25,136
Redundant exact copies with the same label (one copy kept)	−25
Exact copies with conflicting labels (all copies removed)	−79
Near copies with conflicting labels, removed after manual review (25 pairs)	−50
Retained	**24,982**

Table 2: Data quality audit of 30VNFoods version 11.

The 480 confirmed same-label near-duplicate pairs form 412 duplicate groups, which become the split unit in Section 1.4. Of the 54 confirmed cross-label pairs, 25 showed the same picture filed under two dishes and were removed, and 29 showed genuinely different dishes and were kept. Every decision and its reason is recorded in `data/manifests/audit.csv` and `duplicate_review.csv`. Figure 1 shows examples of removed pairs.



Figure 1: *Examples of identical images filed under two different dishes; both copies were removed.*

1.2.3 Bias and Limitations

Nearly half of the source URLs are Instagram, Facebook, or Foody review pages, so styled photographs outnumber everyday phone snapshots. The audit only finds label conflicts between near-identical images; wrong labels on unique images remain. Some classes, such as Banh duc and Banh gio or Bun rieu and Bun mam, are hard to distinguish even for people. Heavily cropped or re-photographed copies can escape the hash checks, so residual leakage cannot be ruled out. The label space covers only 30 dishes, so any other dish is forced into one of them.

1.3 Exploratory Data Analysis

The analysis covers the items required for the classification track: class distribution, imbalance, input size, and representative samples.

1.3.1 Class Distribution and Imbalance

Figure 2 shows the retained images per class and split. Bun bo Hue is the largest class with 1,517 images and Banh pia the smallest with 441, giving an imbalance ratio of $1517/441 = 3.44$. The mean is 833 images per class and the median 797 (coefficient of variation 0.30). The top five classes hold 25.2% of the images and the bottom five hold 10.5%, and the class-distribution entropy reaches 0.988 of its balanced maximum.

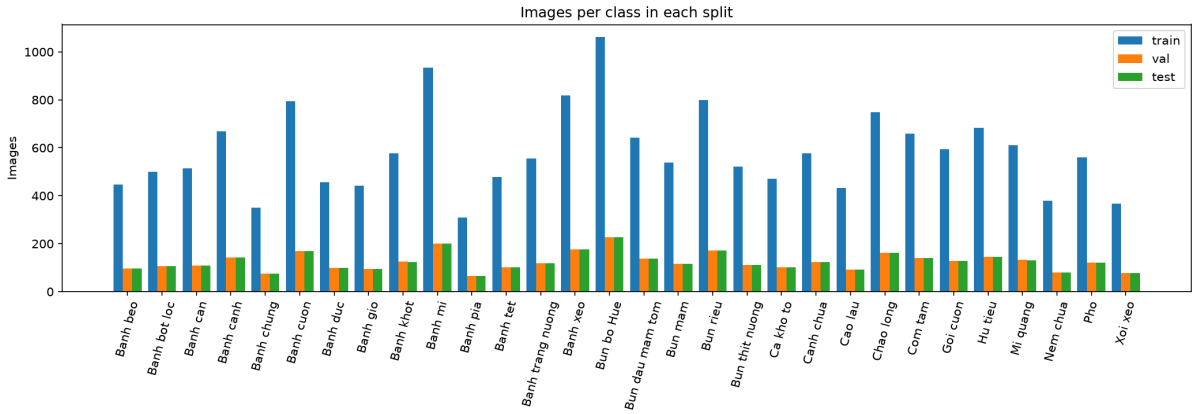


Figure 2: *Retained images per class in the training, validation, and test splits.*

The imbalance is moderate, but it is large enough that accuracy favours the frequent classes. Macro-F1 is therefore the primary metric, because it weights each class equally.[11] The smallest test class has 66 images, so a single error changes its recall by 1.5 percentage points. No run uses class weights or resampling.

1.3.2 Input Size

Image resolutions vary widely (Figure 3). The median image is 1000×720 pixels with a 4:3 aspect ratio. Widths range from 100 to 7,360 pixels and heights from 83 to 5,616 pixels. The dataset contains 14,180 landscape, 6,874 square, and 3,928 portrait images, and 346 images (1.4%) have a short side below 224 pixels and are upscaled during preprocessing. Because sizes span two orders of magnitude, all images are resized and cropped to 224×224.

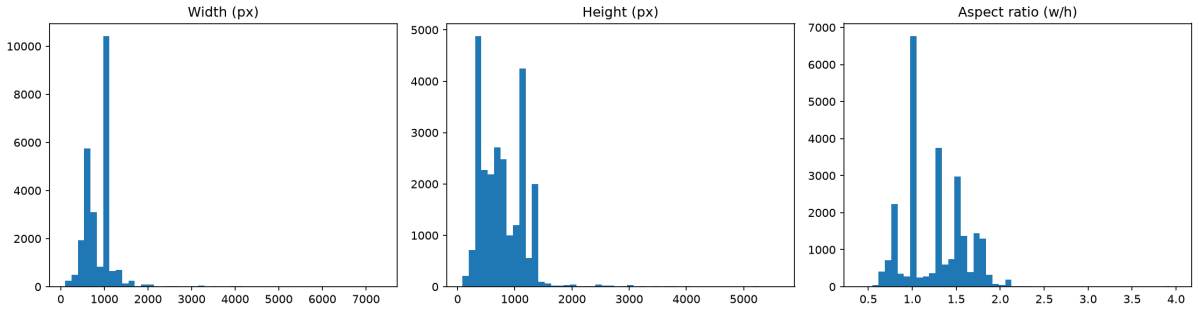


Figure 3: *Distributions of image width, height, and aspect ratio.*

1.3.3 Representative Samples and Expected Difficulties

Figure 4 shows one image per class. Noodle soups share bowls and broths; Banh chung, Banh tet, and Banh gio share banana-leaf wrapping; and griddle cakes such as Banh khot and Banh can come from similar moulds. To make the expected confusions explicit, the 30 classes are grouped into five visual families: noodle bowls (10 classes), steamed and leaf-wrapped cakes (7), griddle cakes (4), rice dishes (3), and breads, rolls, and others (6). The model never sees these families; they are used only to organize the error analysis in Section 3.5.

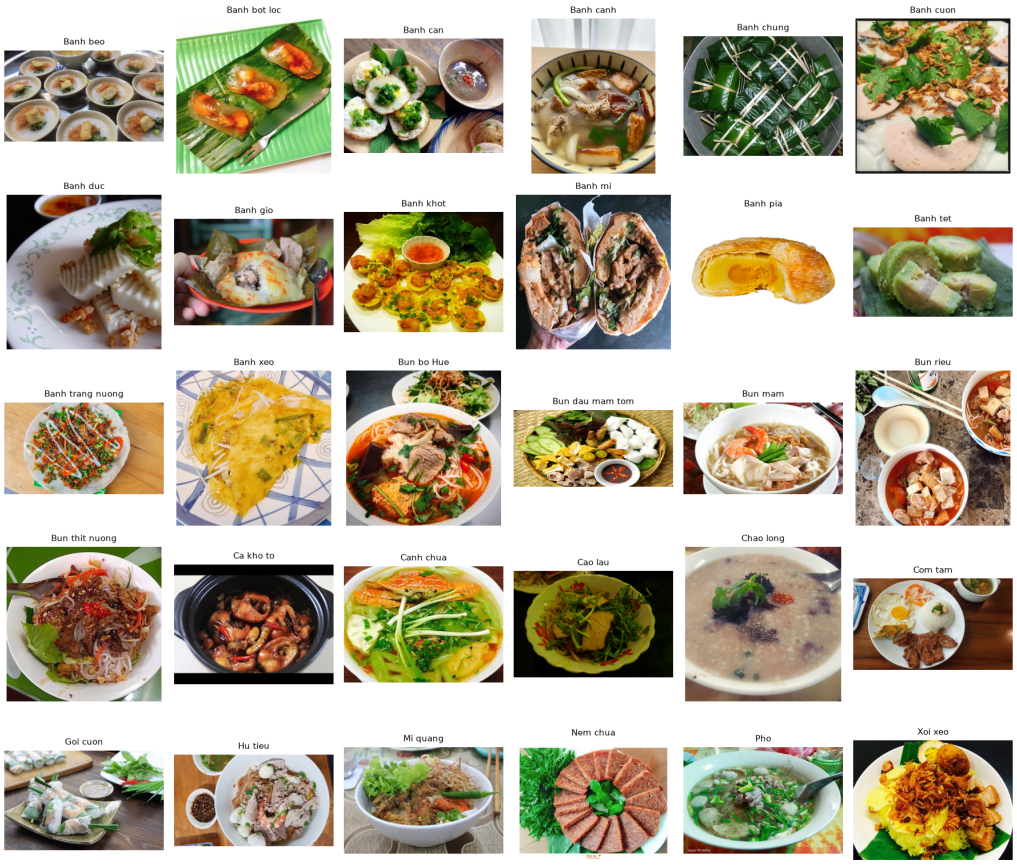


Figure 4: *One representative image from each of the 30 food classes.*

1.4 Data Splitting

The official package split is not used, because 197 duplicate groups and 40 identical files cross its folders. All retained images are pooled and re-split into approximately 70% training, 15% validation, and 15% test with seed 42. The split unit is the duplicate group: an image with no detected match forms its own group, and all members of a group are assigned to the same split. Within each class, groups are shuffled with seed 42; the test split is filled to about 15%, then the validation split to about 15%, and the remaining groups go to training. This keeps class proportions approximately stratified (Table 3). No subset rules apply: all 30 classes and all 24,982 retained images are used.

Split	Images	Groups	Share	Purpose
Training	17,486	17,161	70.0%	Parameter learning
Validation	3,749	3,685	15.0%	Early stopping, checkpoint and experiment selection
Test	3,747	3,688	15.0%	Final evaluation, once per run
Total	24,982	24,534	100%	

Table 3: The shared group-aware split used by all runs.

Leakage checks on the final manifests find no shared image identifiers, duplicate groups, or SHA-256 hashes between any pair of splits. Normalization uses fixed ImageNet statistics, so no statistic is estimated from validation or test images. The validation split selects checkpoints and decides the controlled experiment; the test split is evaluated once per run after the checkpoint is fixed.[11] The manifests are versioned in the repository, and every run records their SHA-256 hashes so that all reported results can be traced to the same split.

1.5 Preprocessing

1.5.1 Transforms

Folder names are mapped alphabetically to class indices 0–29 and stored in `class_to_idx.json`. Each image is decoded with Pillow by content and converted to RGB. Table 4 lists the transforms. Evaluation preprocessing matches the transforms tied to the TorchVision `ResNet50_Weights.IMAGENET1K_V2` checkpoint, as pretrained models assume a specific crop and normalization.[11] Both models use the same transforms so that the comparison is not affected by preprocessing.

Applies to	**Transforms**
Training	`RandomResizedCrop(224, scale=(0.7, 1.0))`, `RandomHorizontalFlip()`, `ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2)`
Validation / test	`Resize(232)` on the shorter side, `CenterCrop(224)`
All splits	`ToTensor()`, `Normalize` with ImageNet mean $(0.485, 0.456, 0.406)$ and standard deviation $(0.229, 0.224, 0.225)$

Table 4: Preprocessing and augmentation applied to each split.

Augmentation is applied to training images only. The crop scale of 0.7–1.0 keeps most of the dish in view, horizontal flips do not change a dish's identity, and mild colour jitter simulates lighting variation. No further cleaning is applied beyond the audit; invalid annotations were handled by removing conflicting copies (Table 2).

1.5.2 Example Batch After Preprocessing

A batch contains images of shape $[64, 3, 224, 224]$ (float32) and labels of shape $[64]$ (int64), and the models return logits of shape $[64, 30]$. The training loader shuffles with a seeded generator, and each worker is seeded so that augmentation is reproducible. Incomplete final batches are kept so that every image is used. Figure 5 shows training images after augmentation, de-normalized for display.

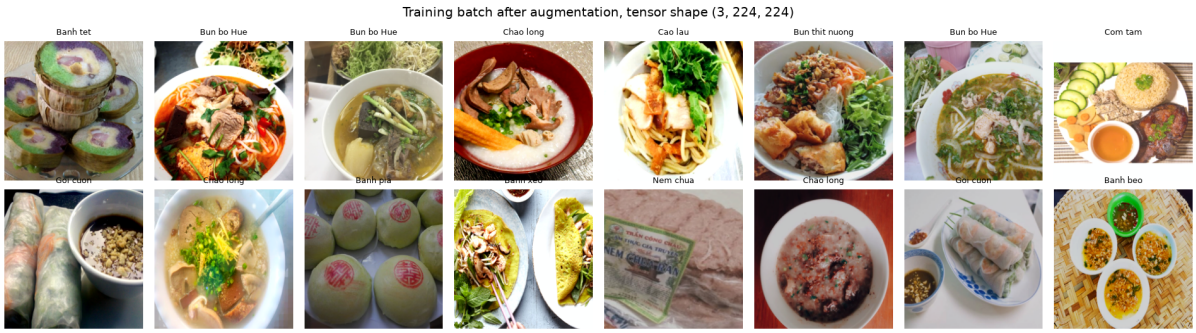


Figure 5: *Training images after augmentation, de-normalized for display and titled with their labels.*

2 Methodology

2.1 Pipeline Overview

Figure 6 summarizes the workflow shared by all runs. The reviewed manifests define the splits, so loaders never regenerate them. Only the model and its trainable part change between runs.

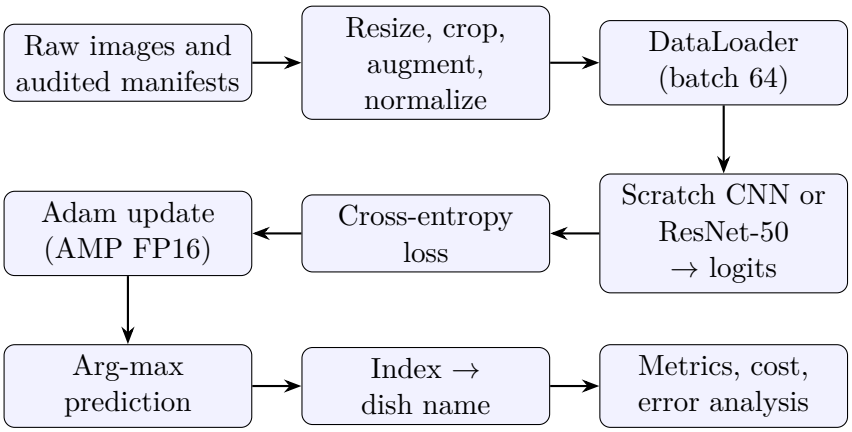


Figure 6: *The shared pipeline from raw images to evaluation.*

During each training batch, the model produces 30 logits, cross-entropy measures the prediction error, and backpropagation supplies gradients for Adam. Validation runs after every epoch in evaluation mode, in FP32, and without gradients. For final evaluation, the selected checkpoint produces logits for the test images, arg-max converts them to class indices, and post-processing maps each index back to a dish name with `class_to_idx.json`. No other post-processing is applied.

2.2 Baseline and Main Methods

All models take $[B, 3, 224, 224]$ inputs and return $[B, 30]$ raw logits. Table 5 summarizes the three compared configurations.

	Scratch CNN (baseline)	Frozen ResNet-50	Full ResNet-50
Architecture	33 plain convolutions (stem and 4 stages), global average pooling, 512 → 256 → 30 head	Bottleneck residual blocks, global average pooling, `Linear(2048, 30)` head	
Initialization	Random	Backbone: `IMAGENET1K_V2`; head: random	
Trained part	All layers	Head only	All layers
Total parameters	21,249,886	23,569,502	23,569,502
Trained parameters	21,249,886	61,470	23,569,502

Table 5: Architecture, initialization, and parameter counts of the three configurations.

2.2.1 Scratch CNN Baseline

The baseline is a plain, VGG-style CNN implemented by the group. Each convolution uses a 3×3 kernel (the stem uses 7×7), omits the bias, and is followed by BatchNorm and ReLU. For unit dilation, each convolution produces the output size[4]

$$H_{\text{out}} = \left\lfloor \frac{H_{\text{in}} + 2p - k}{s} \right\rfloor + 1, \tag{3}$$

where k is the kernel size, s the stride, and p the padding; widths follow the same rule. The network follows the stem-and-stages layout of ResNet in the lectures, without the shortcuts:[6] a stride-2 7×7 stem and max pooling are followed by four stages, and stages 2–4 each start with a stride-2 convolution, so the input is reduced as

$$3\times224\times224 \to 64\times112\times112 \to 64\times56\times56 \to 128\times28\times28 \to 256\times14\times14 \to 512\times7\times7. \tag{4}$$

The stem has one convolution, and stages 1–4 have 6, 8, 12, and 6 (33 in total). Global average pooling produces a 512-dimensional vector, and a two-layer head $512 \to 256 \to 30$ with ReLU produces the logits. All 21,249,886 parameters are trained from random weights.

2.2.1.1 Parameter count. Following the lectures, a convolution with C_{out} filters of size $C_{\text{in}} \times k \times k$, a BatchNorm layer over C channels, and a linear layer from N inputs to M outputs have[7, 8, 9]

$$\#\text{params}_{\text{conv}} = C_{\text{out}}\times C_{\text{in}}\times k\times k + C_{\text{out}}, \qquad \#\text{params}_{\text{BN}} = 2C, \qquad \#\text{params}_{\text{linear}} = M\times N + M. \tag{5}$$

The bias term C_{out} is dropped for every convolution here: each convolution is followed by BatchNorm, whose learnable shift makes a convolution bias redundant, so the code sets `bias=False`.[8] The $2C$ BatchNorm parameters are the scale γ and shift β; the running mean and variance are buffers, not parameters. Pooling, ReLU, and flattening have no

parameters. Table 6 applies these formulas stage by stage; the sum equals the count reported by PyTorch.

Stage	Layers	Output	Conv / linear	BN	Parameters
Stem	7×7 conv $3 \to 64$ (s2); max pool (s2)	64×56^2	9,408	128	9,536
Stage 1	$6\times$ [3×3 conv $64 \to 64$]	64×56^2	221,184	768	221,952
Stage 2	3×3 conv $64 \to 128$ (s2); $7\times$ [$128 \to 128$]	128×28^2	1,105,920	2,048	1,107,968
Stage 3	3×3 conv $128 \to 256$ (s2); $11\times$ [$256 \to 256$]	256×14^2	6,782,976	6,144	6,789,120
Stage 4	3×3 conv $256 \to 512$ (s2); $5\times$ [$512 \to 512$]	512×7^2	12,976,128	6,144	12,982,272
Pool	Global average pooling, flatten	512	–	–	0
Head	Linear $512 \to 256$, ReLU, Linear $256 \to 30$	30	139,038	–	139,038
Total	33 conv, 33 BN, 2 linear layers		21,234,654	15,232	**21,249,886**

Worked counts: stage 1 $= 6 \times (64 \times 64 \times 3 \times 3) + 6 \times 128$; stage 4 $= 512 \times 256 \times 3 \times 3 + 5 \times (512 \times 512 \times 3 \times 3) + 6 \times 1024$; head $= (256 \times 512 + 256) + (30 \times 256 + 30)$. "s2" denotes stride 2.

Table 6: Layer-by-layer architecture and parameter count of the scratch CNN (output shapes exclude the batch dimension).

Stages 3 and 4 hold 93% of the parameters (stage 4 alone 61%), because a 3×3 convolution grows with C^2: one $512 \to 512$ layer has 2,359,296 weights, 64 times more than a $64 \to 64$ layer. The head is small (0.7%) because global average pooling removes the spatial dimensions before the first linear layer.

The baseline uses no external data, so it measures what the training set alone can provide. Its main limitation is optimization: very deep plain networks without shortcut connections are difficult to train,[6] and 17,486 images are few for learning food features from scratch.

2.2.2 Pretrained ResNet-50

The main model is TorchVision ResNet-50 with `ResNet50_Weights.IMAGENET1K_V2`.[13] A residual block learns a correction $F(x)$ to its input instead of the full mapping:[6]

$$y = F(x) + x. \tag{6}$$

The shortcut lets gradients pass through many layers, so deep stacks remain trainable. ResNet-50 uses bottleneck blocks (1×1, 3×3, 1×1 convolutions) in four stages that end at $2048 \times 7 \times 7$, followed by global average pooling. The original 1000-class ImageNet head is replaced by `Linear(2048, 30)`, which has $2048 \times 30 + 30 = 61{,}470$ parameters.[6]

2.2.2.1 Bottleneck block and parameter count. A bottleneck block reduces the channels with a 1×1 convolution, applies a 3×3 convolution at the reduced width C_{mid}, and restores the channels with a second 1×1 convolution. The lectures give its convolution parameters as[10]

$$\#\text{params}_{\text{bottleneck}} = C_{\text{in}} \times C_{\text{mid}} + C_{\text{mid}} \times C_{\text{mid}} \times k^2 + C_{\text{mid}} \times C_{\text{out}}. \tag{7}$$

In ResNet-50 each convolution is followed by BatchNorm and has no bias, and the input is added to the output before the final ReLU. Inside a stage, a block has $C_{\text{mid}} = w$ and $C_{\text{in}} = C_{\text{out}} = 4w$ with $k = 3$, so adding the BatchNorm parameters gives

$$\#\text{params}_{\text{block}} = \underbrace{4w^2 + 9w^2 + 4w^2}_{\text{convolutions}} + \underbrace{2(w + w + 4w)}_{\text{BatchNorm}} = 17w^2 + 12w. \tag{8}$$

For $w = 64$ this gives $17 \times 4096 + 768 = 70{,}400$. When a block changes the shape of its input, the shortcut needs a 1×1 projection convolution,[6] here followed by BatchNorm. This happens in the first block of each stage, and stages 2–4 also halve the resolution with stride 2. Table 7 uses the stage names of the lectures; TorchVision names these modules `conv1` and `layer1`–`layer4`. The sum equals the PyTorch count.

Stage	Layers	Blocks	Output	Parameters
Stem (`conv1`)	Conv 7×7, $3 \to 64$, stride 2, BN; max pool 3×3, stride 2	–	$64 \times 56 \times 56$	9,536
Stage 1 (`layer1`)	Bottleneck, $w = 64$, output 256 channels	3	$256 \times 56 \times 56$	215,808
Stage 2 (`layer2`)	Bottleneck, $w = 128$, output 512 channels	4	$512 \times 28 \times 28$	1,219,584
Stage 3 (`layer3`)	Bottleneck, $w = 256$, output 1,024 channels	6	$1024 \times 14 \times 14$	7,098,368
Stage 4 (`layer4`)	Bottleneck, $w = 512$, output 2,048 channels	3	$2048 \times 7 \times 7$	14,964,736
Pool	Global average pooling	–	2048	0
Head (`fc`)	Linear $2048 \to 30$ (replaces the 1000-class head)	–	30	61,470
Total	Pretrained backbone 23,508,032 + new head 61,470	16		**23,569,502**

Table 7: Stage-by-stage architecture and parameter count of ResNet-50 with the 30-class head (output shapes exclude the batch dimension).

The original ImageNet model has 25,557,032 parameters, of which 2,049,000 belong to its $2048 \to 1000$ head; replacing the head removes 1,987,530 parameters. Stage 4 alone holds 63% of the parameters. In the frozen regime only the head is trained, so 61,470 of 23,569,502 parameters (0.26%) receive gradients; full fine-tuning trains all of them. ResNet-50 has about 2.3 million more parameters than the scratch CNN, but its 1×1 convolutions keep the cost of its 49 main-path convolutions low, and its shortcuts make the deeper network trainable.

ImageNet pretraining provides features for textures, shapes, and object parts, which usually gives faster convergence and better performance on medium-sized datasets.[6] The limitation is that ImageNet contains no Vietnamese dish classes, so its features may not separate the fine-grained cues that distinguish similar dishes.

2.2.3 Fine-tuning Procedure

The two ResNet-50 regimes follow the transfer-learning workflow of loading a pretrained backbone, replacing the head, and optionally unfreezing the backbone with a small learning rate:[6]

- **Frozen backbone (linear probing).** All backbone parameters have `requires_grad = False`, and the backbone stays in evaluation mode during training, so its BatchNorm layers keep their ImageNet running statistics.[5] Only the new head is trained.
- **Full fine-tuning.** All layers are trainable, and BatchNorm statistics adapt to the food images.[12]

Adam receives only trainable parameters, with learning rate 10^{-4}. This is ten times lower than the scratch CNN's rate, to limit early drift away from the pretrained weights. A single constant rate is used without warm-up or layer-wise rates. Both regimes start from the same pretrained weights and the same seeded head initialization, and they share data, augmentation, and epoch budget with the baseline.

2.2.4 Loss and Metrics

All models minimize the cross-entropy loss defined in Section 1, without class weights or label smoothing. Precision, recall, and F1 are computed per class in a one-versus-rest manner:[3]

$$\text{Precision} = \frac{TP}{TP + FP}, \qquad \text{Recall} = \frac{TP}{TP + FN}, \qquad \text{F1} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}. \tag{9}$$

Macro-F1, the primary metric, averages the 30 class scores with equal weight:[3]

$$\text{MacroF1} = \frac{1}{K} \sum_{k=1}^{K} \text{F1}_k, \qquad \text{Accuracy} = \frac{1}{N} \sum_{i=1}^{N} \mathbf{1}(\hat{y}_i = y_i). \tag{10}$$

Accuracy is the secondary metric. Per-class precision, recall, and F1, the confusion matrix, and prediction examples support the error analysis. An undefined class F1 contributes zero.

2.2.5 Implementation Components

The group implemented the data audit and group-aware split, the manifest dataset class, the scratch CNN, the ResNet-50 wrapper with the freezing logic, the training loop (mixed precision, early stopping, resumable checkpoints), checkpoint selection and reload verification, the evaluation and error-analysis scripts, the inference benchmark, and the plots. The following libraries supply the underlying operations:

- **PyTorch:** layers, Adam, cross-entropy, `autocast` and `GradScaler`, and `DataLoader`.
- **TorchVision:** the ResNet-50 architecture and `IMAGENET1K_V2` weights, and image transforms.
- **scikit-learn:** accuracy, precision, recall, F1, and the confusion matrix.
- **Pillow and NumPy:** image decoding, hashing, and array operations.

2.3 Training Setup

2.3.1 Shared Hyperparameters

Table 8 lists the settings shared by all runs. Only the learning rate differs between the configurations.

Setting	Value
Loss	Mean multiclass cross-entropy on raw logits
Optimizer	Adam, weight decay 0
Learning rate	10^{-3} (scratch CNN); 10^{-4} (both ResNet-50 runs); constant, no scheduler
Batch size	64
Maximum epochs	20
Early stopping	Patience 5 on validation macro-F1
Checkpoint selection	Highest validation macro-F1; ties keep the earlier epoch
Regularization	Data augmentation only; no weight decay, dropout, or label smoothing
Random seed	42 for Python, NumPy, PyTorch, and the loader generator
Precision	AMP FP16 training with `GradScaler`; FP32 validation and test
Data loading	4 workers, persistent workers, prefetch factor 2, pinned memory

Table 8: Shared training settings used for all three runs.

2.3.2 Early Stopping and Checkpoint Selection

After every epoch, the model is evaluated on the validation split in FP32. `best_model.pt` is overwritten only when validation macro-F1 is strictly higher than at every earlier epoch, so ties keep the earlier checkpoint. Early stopping monitors the same quantity: training ends after five consecutive epochs without a new best validation macro-F1, or when the 20-epoch budget is reached. Because both rules use one metric, the stopped run always reports its best epoch, and a resumed run continues with the same counter. No run stopped early.

After training, the selected checkpoint is reloaded, and its validation accuracy and macro-F1 must reproduce the saved values exactly before the test split is evaluated. All three runs passed this check (`matches_saved_metrics: true`).

2.3.3 Hyperparameter Selection

No hyperparameter search was run. The learning rates follow common practice for training from scratch (10^{-3}) and for fine-tuning a pretrained backbone (10^{-4}), and the remaining settings are shared. The comparison therefore evaluates one reasonable configuration per model under a common budget, rather than the best achievable configuration of each model.

2.3.4 Experimental Environment and Training Time

The final runs used one Kaggle Tesla T4 GPU (14.6 GB available) with 4 CPU cores, Python 3.13.16, PyTorch 2.11.0+cu130, TorchVision 0.26.0+cu130, CUDA 13.0, cuDNN 9.27.0, NumPy 2.1.3, pandas 2.3.3, and scikit-learn 1.6.1. Training used channels-last memory format and cuDNN benchmark mode, so reruns are fast but not bit-identical; all compared runs used the same options. Each run folder stores its resolved configuration, manifest hashes, environment record (`pip_freeze.txt`, `gpu.txt`), epoch history, and an exact source snapshot.

The earlier M2 runs used a local RTX 5070 Ti in FP32 without early stopping. They are not compared with the final runs, because hardware and precision differ. Table 9 reports the elapsed time of the epoch loop, which includes training, validation, data loading, and checkpoint saving but excludes setup and weight download.

Run	Run ID	Epochs	Training time
Scratch CNN	`20261103T145419Z_7a003524`	20	65 min 13 s
Frozen ResNet-50	`20261103T160427Z_d37a574f`	20	55 min 28 s
Full ResNet-50	`20261103T170311Z_ce0f6099`	20	60 min 28 s

Table 9: Run identifiers, completed epochs, and training times of the three final runs.

2.4 Experimental Design

Table 10 states both experiments. Both decisions use validation macro-F1 only. The test split is scored once per run after the checkpoint is fixed and is reported as confirmation; no model or hyperparameter was changed because of test results.

	A: baseline versus pretrained	**B: frozen versus full fine-tuning (controlled)**
Hypothesis	ImageNet features transfer to food photographs, so a pretrained ResNet-50 outperforms a CNN trained from scratch on about 17,500 images.	Adapting the whole backbone to food images improves over a fixed ImageNet feature extractor.
Changed factor	Architecture and initialization (scratch CNN versus frozen ResNet-50); the learning rate also differs.	`freeze_backbone` only (true versus false).
Fixed factors	Manifests, seed, preprocessing and augmentation, loss, optimizer, batch size, epoch budget, early-stopping rule, AMP setting, and GPU type.	Everything fixed in A, plus the architecture, pretrained weights, seeded head initialization, and learning rate 10^{-4}.
Metrics	Validation and test macro-F1 (primary) and accuracy.	Same, plus trained parameters, training time, and inference cost.
Comparison	Δ validation macro-F1 = frozen − CNN at the selected checkpoints; the gain is not attributed to a single factor.	Δ validation macro-F1 = full − frozen, reported together with the extra cost.

Table 10: Hypotheses, factors, metrics, and comparisons of the two experiments.

Experiment B is the mandatory controlled ablation: the two runs differ only in whether the backbone is frozen. Experiment A changes the model, initialization, and learning rate together, so it shows whether the pretrained model is better, not which factor causes the gain.

2.4.1 Compute-Cost Measurement

Compute cost is reported as total and trained parameters, checkpoint size, training time, and inference cost of each selected checkpoint. Inference is measured in two ways:

- **Model only:** forward passes on synthetic inputs at batch size 1 and 64, in FP32 and AMP FP16, with 10 warm-up and 50 timed iterations and CUDA synchronization. This excludes data loading and isolates model cost.
- **End to end:** one FP32 pass over the real test split, including image decoding, transforms, and host-to-device transfer.

Epoch times also record how long the training loop waits for the data loader, so that model computation and data loading can be separated. All timings are specific to the shared Kaggle T4 and are indicative rather than controlled measurements.

3 Implementation Results

The following results come from the three final runs on the same 3,747-image test set. Each model is evaluated with its highest-validation-macro-F1 checkpoint, which was reloaded and verified before the single test evaluation. All results come from one run per configuration with seed 42.

3.1 Training Behavior

3.1.1 Learning Curves and Selected Checkpoints

Figure 7 plots loss, accuracy, and macro-F1 for every run. The selected epochs are 19 for the scratch CNN, 19 for frozen ResNet-50, and 17 for full ResNet-50. All runs completed the 20-epoch budget: the two runs selected at epoch 19 had no five-epoch plateau, and full fine-tuning set its best value at epoch 17, only three epochs before the budget ended. Training metrics are measured with augmentation enabled, so they are not directly comparable with validation metrics.

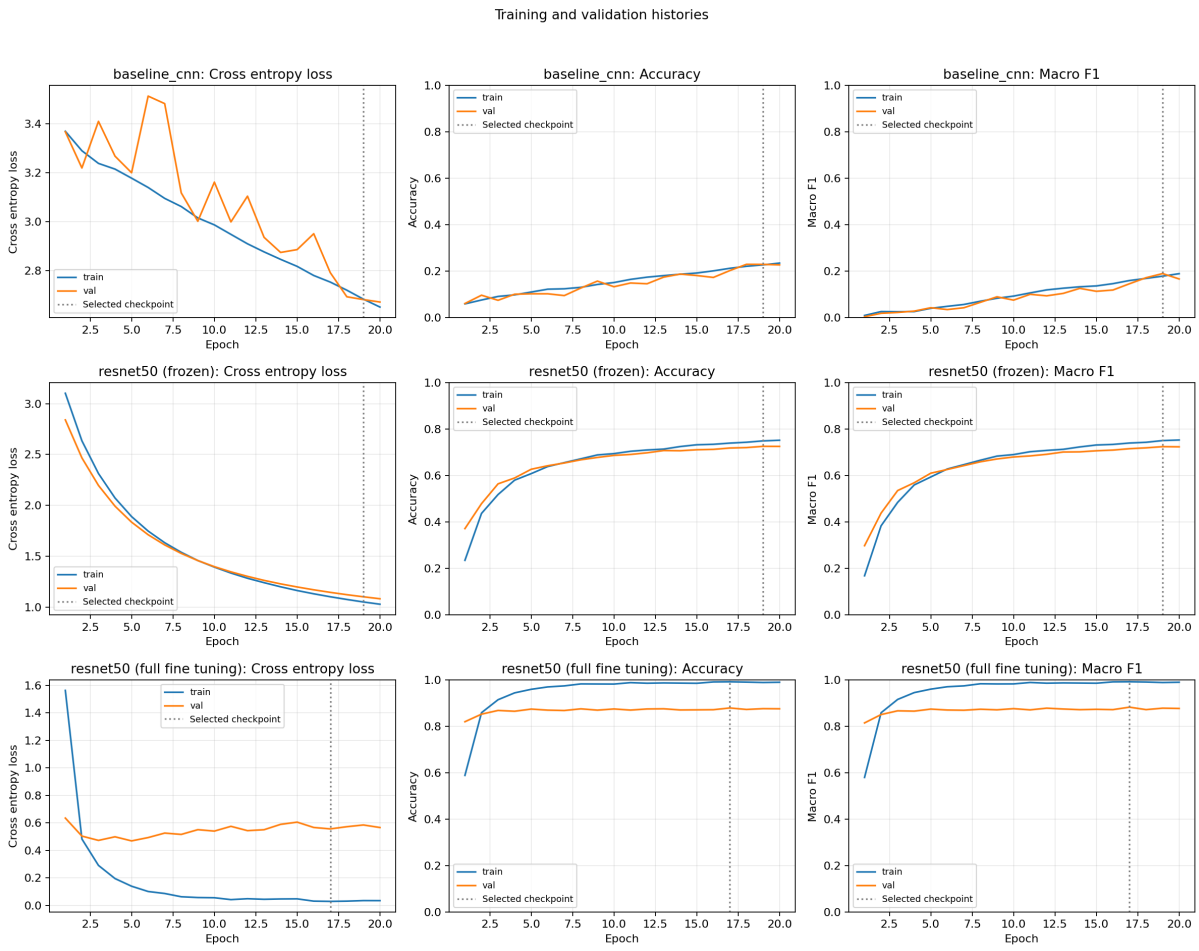


Figure 7: *Training and validation curves of the three final runs; dotted lines mark the selected epochs.*

3.1.2 Underfitting and Overfitting

- **The scratch CNN underfits.** Training and validation accuracy stay close at about 23% (23.5% versus 22.7% at epoch 20), validation loss fluctuates (for example, it rises from 3.22 to 3.41 at epoch 3 and to 3.51 at epoch 6), and both curves are still rising at epoch 20. The model has not converged within the budget.
- **Frozen ResNet-50 converges smoothly.** Validation macro-F1 rises from 0.298 after the first epoch to 0.724 at epoch 19. The train–validation gap stays small (75.2% versus 72.6% accuracy at epoch 20), and validation macro-F1 still gains about 0.004 per epoch near the end, so a longer budget would likely help slightly.
- **Full ResNet-50 overfits after a fast start.** Validation macro-F1 reaches 0.815 after one epoch and plateaus near 0.87 from epoch 3. Training accuracy climbs to 99.2%, while validation loss rises from its minimum of 0.469 at epoch 5 to about 0.57. Validation macro-F1 gains only 0.008 after epoch 5, so most of the benefit is obtained in the first few epochs.

3.1.3 Implementation Issues and Fixes

The local M2 runs showed low GPU utilization and took about 45 minutes each in FP32. For the final runs, the training loop added mixed precision, channels-last memory format, persistent workers with prefetching, early stopping, and resumable checkpoints (`last.pt`), and the pipeline moved to Kaggle. Per-epoch logs show that the loader is still the bottleneck: from epoch 2 onwards, the training loop waits for data for 84–114 seconds of each 155–205-second epoch, and the mean training throughput is 116–128 images per second for all three models, regardless of model size. This matches the symptoms of a data-pipeline bottleneck on the 4-core Kaggle machine.[11] The first CNN epoch took 764.5 seconds, of which 590.1 seconds were spent waiting for data, most likely because worker start-up and the first read of the mounted dataset fell into it. Validation runs in FP32 so that the reload check is exact, and the runner refuses to evaluate the test split when the reloaded checkpoint does not reproduce its saved validation metrics.

3.2 Quantitative Results

3.2.1 Predictive Performance

Table 11 reports validation and test performance. Full ResNet-50 reaches the best results on every metric: test macro-F1 0.8787 and test accuracy 87.80%, correctly classifying 3,290 of 3,747 test images.

Model	Epoch	Val Acc	Val F1	Test Acc	Test F1	Test errors
Scratch CNN	19	22.94%	0.1898	23.30%	0.1885	2,874
Frozen ResNet-50	19	72.58%	0.7245	71.92%	0.7179	1,052
Full ResNet-50	17	**87.89%**	**0.8824**	**87.80%**	**0.8787**	**457**

Table 11: Validation and test performance of the selected checkpoints (F1 denotes macro-F1).

Validation and test macro-F1 differ by less than 0.007 for every model, so selecting the checkpoint on validation data did not overfit the validation split. Accuracy and macro-F1 stay within 0.01 of each other for both ResNet-50 runs, which indicates that their errors are not concentrated in the small classes. With one run per configuration, mean and standard deviation are not reported.

3.2.2 Compute Cost

Table 12 combines training and inference cost on the Tesla T4.

Measure	Scratch CNN	Frozen	Full
Total parameters	21,249,886	23,569,502	23,569,502
Trained parameters	21,249,886	61,470	23,569,502
Checkpoint size (MB)	243.4	90.7	270.2
Training time (min)	65.2	55.5	60.5
Mean epoch time, epoch 2 on (s)	165.7	165.9	180.6
Training throughput (images/s)	127.4	127.7	116.4
Latency, batch 1, FP32 (ms/image)	8.73	8.25	8.27
Latency, batch 1, AMP FP16 (ms/image)	3.24	6.73	5.88
Throughput, batch 64, FP32 (images/s)	324	242	243
Throughput, batch 64, AMP FP16 (images/s)	1,534	807	804
Peak memory, batch 64, FP32 (MB)	521	777	777
End-to-end test throughput, FP32 (images/s)	22.3	102.3	101.3

Table 12: Training and inference cost on one Kaggle Tesla T4 (Frozen and Full denote the two ResNet-50 regimes).

Both ResNet-50 variants share one architecture, so their inference cost is the same within measurement noise. Their checkpoint sizes differ because each file also stores the Adam state of the trained parameters. The scratch CNN is the fastest model at batch 64 with AMP, about 1.9 times the ResNet-50 throughput. Its end-to-end test throughput of 22.3 images per second contradicts the model-only benchmark, so it reflects data loading rather than the model. The most likely cause is that its evaluation was the first to read the test images from the mounted dataset, before the file cache was warm; this was not profiled.

3.3 Qualitative Results

3.3.1 Correct and Incorrect Predictions

Figure 8 shows test predictions of full ResNet-50 in three groups: the most confident correct predictions, the correct predictions with the lowest confidence, and the most confident errors. These examples are selected by confidence and illustrate behaviour; they are not a representative sample.

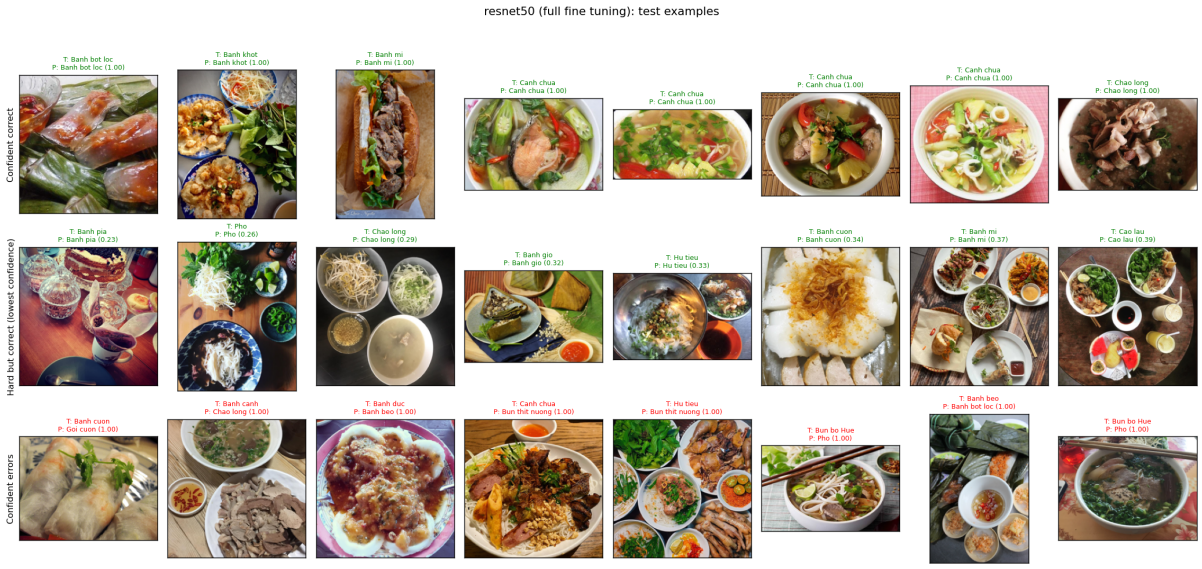


Figure 8: *Confident correct, low-confidence correct, and confident wrong test predictions of full ResNet-50 (T = true class, P = predicted class, with softmax confidence).*

Confident correct predictions are close-up photographs of a single dish. Low-confidence correct predictions are mostly table scenes with several dishes, where the labelled dish is recognized but competes with others for probability. Confident errors fall between visually similar dishes, such as Bun bo Hue and Pho or Banh cuon and Goi cuon, and on mixed platters where another dish dominates the frame. These are plausible visual explanations, not a verified account of the model's internal reasoning.

3.4 Comparison and Discussion

3.4.1 Hypothesis Assessment

Both hypotheses are supported (Table 13). Pretraining adds 0.53 test macro-F1 over the scratch CNN, and full fine-tuning adds another 0.16 over the frozen backbone. Each gain is the macro-F1 of the second model minus that of the first.

Experiment	Val gain	Test gain	Verdict
A: CNN → frozen ResNet-50	+0.5347	+0.5294	Supported, but not attributable to a single factor
B: frozen → full ResNet-50	+0.1579	+0.1608	Supported; full fine-tuning is better on all 30 classes

Table 13: Macro-F1 gains and verdicts for the two experiments.

3.4.2 Where the Differences Appear

Table 14 lists per-class test F1. Full fine-tuning improves all 30 classes over the frozen backbone. The largest gains are on noodle soups and lookalike cakes: Banh duc +0.39 (0.36 to 0.75), Bun rieu +0.31, Bun mam +0.30, Hu tieu +0.30, Mi quang +0.27, and

Pho +0.25. Distinctive dishes gain little: Banh mi, Banh beo, Ca kho to, and Bun dau mam tom already exceed 0.82 F1 with the frozen backbone and gain at most 0.05.

The scratch CNN collapses toward the largest class. Its ten most frequent confusions include eight that predict Bun bo Hue, for example 84 of the 171 Bun rieu test images (49.1%), and five classes (Banh can, Banh canh, Banh duc, Bun mam, and Cao lau) have zero F1. Most of its errors are uncertain: 2,845 of its 2,874 errors have confidence below 0.5.

Table 14: Per-class test F1 of the scratch CNN and the frozen and fully fine-tuned ResNet-50, with test support.

Class	Scratch CNN	Frozen	Full	Support
Banh beo	0.220	0.828	0.850	96
Banh bot loc	0.029	0.640	0.811	107
Banh can	0.000	0.758	0.884	110
Banh canh	0.000	0.492	0.705	143
Banh chung	0.268	0.761	0.907	75
Banh cuon	0.282	0.767	0.888	170
Banh duc	0.000	0.362	0.747	98
Banh gio	0.118	0.811	0.927	95
Banh khot	0.120	0.753	0.863	124
Banh mi	0.541	0.926	0.952	200
Banh pia	0.379	0.850	0.917	66
Banh tet	0.162	0.840	0.923	102
Banh trang nuong	0.389	0.862	0.946	119
Banh xeo	0.494	0.818	0.927	176
Bun bo Hue	0.297	0.634	0.804	228
Bun dau mam tom	0.212	0.913	0.961	137
Bun mam	0.000	0.587	0.887	116
Bun rieu	0.071	0.523	0.834	171
Bun thit nuong	0.048	0.652	0.842	112
Ca kho to	0.270	0.898	0.939	101
Canh chua	0.104	0.648	0.885	123
Cao lau	0.000	0.784	0.877	92
Chao long	0.252	0.705	0.948	161
Com tam	0.192	0.792	0.968	141
Goi cuon	0.279	0.878	0.954	128
Hu tieu	0.208	0.473	0.770	146
Mi quang	0.137	0.630	0.899	131
Nem chua	0.088	0.688	0.829	81
Pho	0.236	0.526	0.777	120
Xoi xeo	0.260	0.735	0.938	78

3.4.3 Likely Causes

- **Data scale and initialization.** About 17,500 images and 20 epochs are not enough for a 21-million-parameter plain CNN to learn food features from random weights,

and its deep stack without shortcuts is hard to optimize. ImageNet features already encode textures, shapes, and bowls.

- **Fixed features limit the frozen model.** Noodle soups differ mainly in broth colour, noodle shape, and toppings. Generic ImageNet features keep them close together, so a linear head cannot separate them well.
- **Backbone adaptation drives the full model.** Updating every layer, including the BatchNorm statistics, lets the features learn these fine-grained cues. Without weight decay, the model also memorizes the training set (99% training accuracy).
- **Class imbalance is not the main factor.** For both ResNet-50 runs, accuracy and macro-F1 stay within 0.01 of each other, and the weakest classes (Banh canh, Banh duc, Hu tieu, and Pho) are not the smallest ones.

3.4.4 Accuracy–Speed–Parameter Trade-offs

Full fine-tuning is inexpensive relative to its gain. It trains 383 times more parameters than the frozen model, but takes only 9% longer to train (60.5 versus 55.5 minutes), because both regimes pass every image through the full backbone and data loading dominates the epoch time. Inference speed is identical; only the checkpoint grows (270 versus 91 MB). The scratch CNN has the highest inference throughput but reaches only 0.19 macro-F1, so its speed has no practical value. It also took the longest to train (65 minutes), mostly because of its slow first epoch, which again points to data loading rather than model computation.

3.4.5 Conclusions Not Yet Supported

All results come from one seed, so run-to-run variance is unknown, and small differences, such as the 0.008 macro-F1 gained after epoch 5 by full fine-tuning, are not meaningful. Experiment A changes model, initialization, and learning rate together, so it cannot show how much of the gain comes from pretraining alone. A longer CNN schedule, or full fine-tuning with weight decay and earlier stopping, could change the margins; neither was tested. The timing results come from a shared Kaggle GPU and are indicative.

3.5 Error Analysis

Full ResNet-50 misclassifies 457 of 3,747 test images (12.2%). Figure 9 shows its row-normalized confusion matrix: errors concentrate in a few off-diagonal blocks that correspond to the noodle-soup and steamed-cake families.

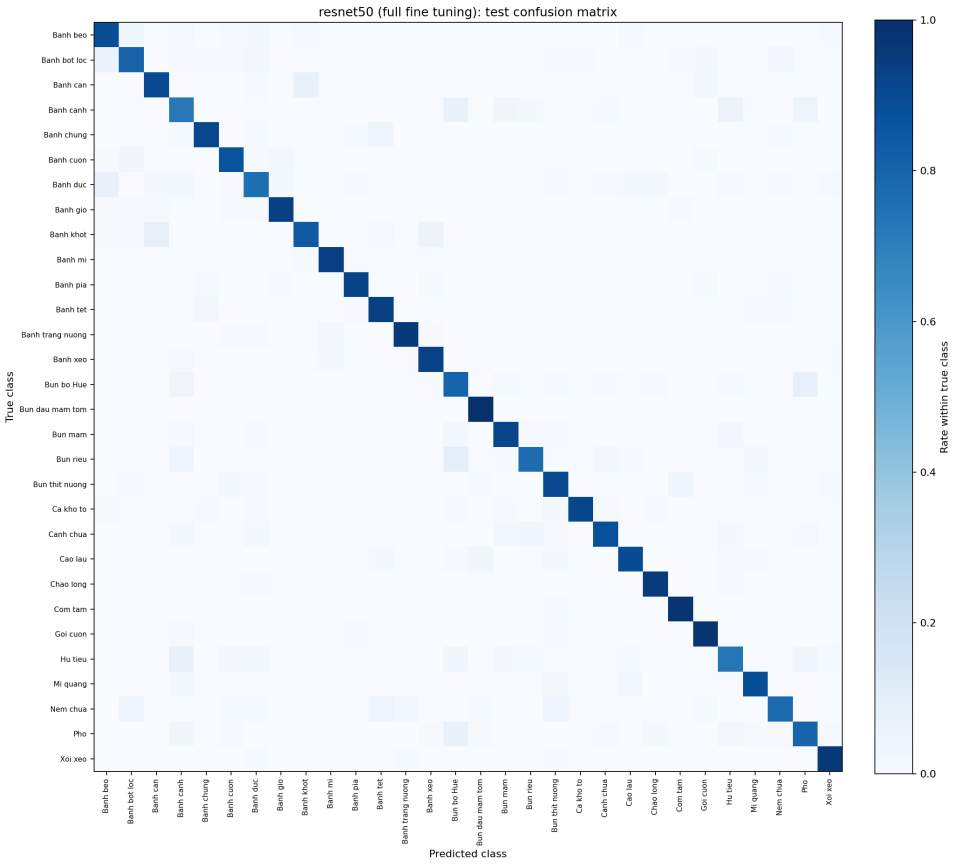


Figure 9: *Test confusion matrix of full ResNet-50, with true classes in rows and each row normalized by the class support.*

3.5.1 Error Groups

Table 15 groups the 457 errors by the visual families of the true and predicted classes (Section 1.3). Four in five errors are either noodle-soup confusions or confusions across families. By true family, the error rate is highest for noodle soups (16.9%) and steamed cakes (13.2%), and lowest for rice dishes (3.4%), which have no within-family errors at all.

Error group	Errors	Most frequent pairs (true → predicted)	Hypothesized cause	Improvement idea
Noodle soups	185 (40.5%)	Bun bo Hue → Pho 17; Bun rieu → Bun bo Hue 15; Hu tieu → Banh canh 10	Bowls share broth, herbs, and plating; noodle thickness and broth colour are small cues at 224 px, often hidden by toppings.	Higher input resolution; more training images for these classes.
Across families	183 (40.0%)	Bun rieu → Canh chua 5; Nem chua → Bun thit nuong 4; Banh xeo → Banh mi 4	Multi-dish scenes, unusual views, and shared components conflict with the single label.	Crop to the main dish before classification; clean doubtful labels.
Steamed cakes	54 (11.8%)	Banh bot loc → Banh beo 7; Banh duc → Banh beo 7; Banh cuon → Banh bot loc 6	White, translucent rice-flour textures and similar plating with sauce and shrimp.	Reduce colour jitter, which may blur translucency cues; add hard examples.
Griddle cakes	28 (6.1%)	Banh khot ↔ Banh can 18; Banh khot → Banh xeo 7	Small round cakes from the same moulds, differing mainly in toppings and size.	Higher resolution; crops that keep scale cues.
Other within family	7 (1.5%)	Single, scattered pairs	No clear pattern.	—

Table 15: Test errors of full ResNet-50 grouped by the dish families of the true and predicted classes.

The confusions are often mutual, which indicates fine-grained visual similarity rather than a one-sided bias: Bun bo Hue and Pho are confused 17 and 8 times in the two directions, Banh canh and Bun bo Hue 10 and 11 times, Banh can and Banh khot 8 and 10 times, and Banh canh and Hu tieu 8 and 10 times. The weakest classes are Banh canh (F1 0.705), Banh duc (0.747), Hu tieu (0.770), and Pho (0.777).

3.5.2 Cross-Family Errors: Sample Review

To understand the 183 cross-family errors, 30 of them were sampled at random (seed 0) and inspected visually (Figure 10). They fall into five groups:

- **Multi-dish or cluttered scenes (8 of 30):** trays, street stalls, and people eating hide the labelled dish, as in Com tam → Bun thit nuong.
- **Unusual view or cooking stage (8 of 30):** close crops, a lidded pot, or a cake still in the pan, as in Banh xeo → Banh pia.
- **Soups from another family (8 of 30):** noodle soups predicted as Canh chua or Chao long, and the reverse.
- **Shared wrapping or components (4 of 30):** banana-leaf packages and grilled meat, as in Nem chua → Banh tet.
- **Doubtful labels (2 of 30):** the image does not appear to show the labelled dish.

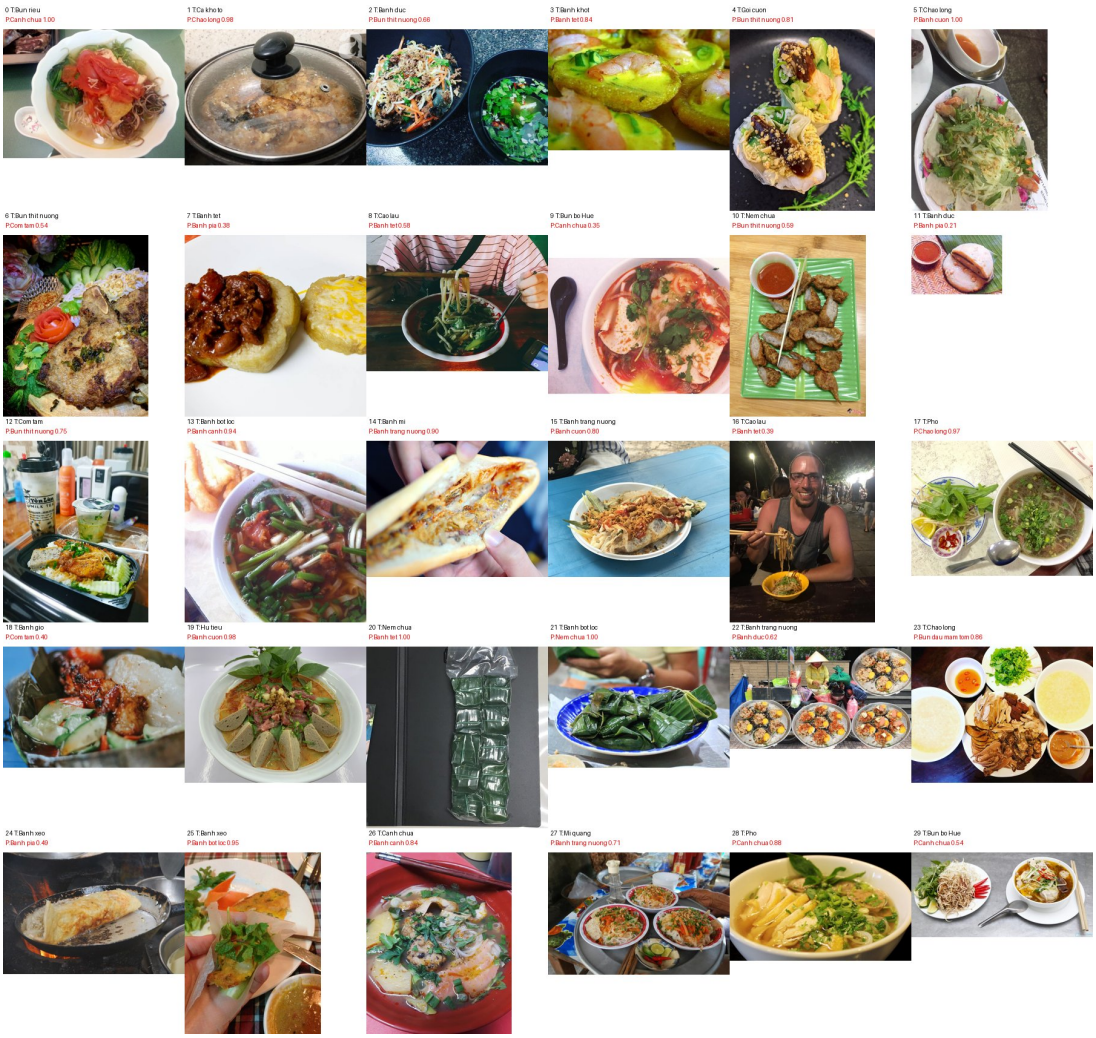


Figure 10: *Thirty randomly sampled cross-family errors of full ResNet-50 (seed 0), with true class, prediction, and confidence.*

3.5.3 Confidence of Errors and Errors Shared by All Models

Table 16 groups errors by confidence. Full ResNet-50 is often confidently wrong: 225 of its errors (49%) have confidence at least 0.8, and only 53 fall below 0.5. Its true class is

still among the top three predictions for 321 errors (70%). Temperature scaling or label smoothing could make confidence usable as a reject signal, and a top-3 output would recover most mistakes in a search or tagging application. In contrast, the frozen model and the scratch CNN make mostly uncertain errors.

Test errors	**Scratch CNN**	**Frozen**	**Full**
Total errors	2,874	1,052	457
Confident (confidence ≥ 0.8)	1	7	225
Moderate (0.5 to 0.8)	28	99	179
Uncertain (< 0.5)	2,845	946	53
True class in top 3	763	661	321
Mean probability of the true class	0.063	0.131	0.094

Table 16: Test errors of the three models grouped by prediction confidence (Frozen and Full denote the two ResNet-50 regimes).

Across the three models, 753 test images are misclassified by none, 1,909 by one, 781 by two, and 304 by all three. The images that every model misclassifies concentrate in Banh canh (32), Bun rieu (27), Hu tieu (26), and Banh duc (21). These shared errors are the most likely to contain ambiguous content or label noise and should be inspected manually before any relabelling.

3.6 Ablation Study

Experiment B is the ablation required for Assignment 2. It removes one component, the update of the pretrained backbone, while keeping the architecture, pretrained weights, seeded head initialization, data, augmentation, optimizer, learning rate, batch size, epoch budget, early-stopping rule, and precision fixed. Table 17 summarizes the result.

Measure	**Frozen**	**Full**	**Full − frozen**
Validation macro-F1	0.7245	0.8824	+0.1579
Test macro-F1	0.7179	0.8787	+0.1608
Validation accuracy	72.58%	87.89%	+15.31 pp
Test accuracy	71.92%	87.80%	+15.88 pp
Trained parameters	61,470	23,569,502	+23,508,032
Training time (min)	55.5	60.5	+5.0
Selected epoch	19	17	−2
Inference, batch 64, AMP (images/s)	807	804	≈ 0

Table 17: Controlled ablation: frozen backbone versus full fine-tuning of ResNet-50.

Full fine-tuning improves validation macro-F1 by 0.1579, and the test gain of 0.1608 confirms it. The extra cost is moderate: 9% more training time, no change in inference speed, and a larger checkpoint. Adapting the backbone is therefore the component that

matters most for this dataset. The conclusion holds for the tested setting (one learning rate, no weight decay, 20 epochs); layer-wise learning rates or gradual unfreezing were not tested.

3.7 Limitations and Conclusion

3.7.1 Limitations

- **Data:** web photographs favour styled review images, and label noise on unique images remains. Heavily edited copies may still leak across splits despite the duplicate audit.
- **Method:** the single-label formulation cannot handle multi-dish photographs, which drive a large share of the cross-family errors. Full fine-tuning overfits without weight decay, and its confidence is poorly calibrated.
- **Experiment:** each configuration was trained once with one split and seed 42, so no confidence intervals are reported, and per-class test F1 rests on 66–228 images per class. The validation split drives early stopping, checkpoint selection, and the experiment decision, so validation scores are slightly optimistic. Experiment A does not isolate a single factor, and the shared 20-epoch budget favours the pretrained models.
- **Generalization:** the model covers only 30 dishes and forces any other food into one of them. The results may not hold for everyday phone photographs or for regional variants missing from the data.

3.7.2 Conclusion and Future Work

ImageNet pretraining is essential at this data scale, and full fine-tuning is clearly worth its cost. Full ResNet-50 reaches test macro-F1 0.8787 and accuracy 87.80%, against 0.7179 for the frozen backbone and 0.1885 for the scratch CNN. The remaining errors concentrate on visually similar noodle soups and steamed cakes and on photographs that contain several dishes. Future work should:

- repeat each configuration over at least three seeds and report mean $\pm$ standard deviation;
- add weight decay and stop near the validation plateau around epoch 5, which would reduce overfitting and training time;
- test a higher input resolution for the noodle-soup and griddle-cake confusions;
- crop to the main dish, calibrate confidence, and return top-3 predictions for application use;
- profile and speed up data loading, which dominates the epoch time on the Kaggle GPU.

4 AI Usage Disclosure

This section summarizes entries A2-1–A2-4 in AI_USAGE.md. Prompt examples are representative summaries, not verbatim transcripts. Code, configuration, test, and notebook paths below are relative to `assignment2/`; website files are relative to the repository root.

4.1 Project Scaffold, Environment, and Experiment Runner

Tool / model Codex (OpenAI), GPT-6, coding-agent session
Member / verifier Tran Manh Thang (2353124)
Stage Project setup and M2 Draft implementation

- **Purpose and scope:** Review of the folder and file skeleton; guidance on Docker and dependency setup, configuration loading and merging, seeding, experiment context, training orchestration, run provenance, checkpoint verification, comparison and plot export, and CI; drafting of the M2 README, report, and page content. Files: `config/`, `experiments/context.py`, `run.py`, `verify_checkpoint.py`, `compare.py`, `plot_curves.py`, `src/utils/`, Docker and CI files, `README.md`, and `report/`.
- **Edits:** Code examples were implemented by hand and adapted to the project modules; copied-code mismatches, stale Docker source, test discovery, and container shared-memory issues were fixed during debugging. No class labels, data, or results were generated by the tool.
- **Verification:** The Docker CPU test suite passed 25 tests; both M2 runs completed on the GPU, and each reloaded checkpoint reproduced its saved validation accuracy and macro-F1. Sources: the Course Project Handbook and PyTorch, TorchVision, Docker, GitHub Actions, and pytest documentation.

Representative prompt example

> "I already created a file structure can you check the type and structure is it correct or not if not you can fix by yourself."

4.2 Data Audit, Splitting, and EDA

Tool / model Claude (Anthropic), Claude Opus 5.5, desktop app
Member / verifier Phan Tan Loc (2352712)
Stage M2 Draft data and EDA development

- **Purpose and scope:** Assistance with the data audit, exact and near-duplicate detection, group-aware splitting, the dataset class, transforms and loaders, and the EDA notebook and figures. Files: `src/data/`, `data/manifests/`, `notebooks/01_eda.ipynb`, EDA figures, `tests/test_dataset.py`, and `tests/test_split.py`.
- **Edits:** Integrated into the shared pipeline. The 54 cross-label near-duplicate pairs were reviewed by hand: 25 were removed as the same picture under two dishes and 29 were kept as different dishes.

- **Verification:** Unit tests check RGB tensors and integer labels, batch shapes, reproducible manifest hashes, the 70/15/15 ratio with full class coverage, seed reproducibility, audit rules, and the absence of shared images or groups across splits. Sources: PyTorch, TorchVision, and Pillow documentation.

Representative prompt example

> "Write an audit for the 30VNFoods images that finds unreadable files, exact copies by SHA-256 and near copies by perceptual hash, then build a 70/15/15 split per class where all copies of an image stay in the same split, with a fixed seed."

4.3 Models, Training, and Evaluation Code

Tool / model	Claude (Anthropic), Claude Opus 5.5, VS Code extension
Member / verifier	Hoang Kim Cuong (2352145)
Stage	M2 Draft model and training development

- **Purpose and scope:** Assistance with the scratch CNN, the ResNet-50 wrapper and factory, the epoch trainer, the checkpoint helper, the metrics and evaluator, and the initial error-analysis module. Files: `src/models/`, `src/training/`, `src/evaluation/metrics.py`, `evaluator.py`, `error_analysis.py`, and the matching tests.
- **Edits:** Connected to the shared configuration and the experiment runner; the models return raw logits, and checkpoint selection keeps the earlier epoch on ties.
- **Verification:** Unit tests in `tests/test_models.py`, `test_metrics.py`, and `test_checkpoint.py` check output shapes, the backward pass, metrics on known predictions, per-class metrics when some classes are never predicted, the tie rule, and checkpoint restoration. Every reloaded checkpoint reproduced its saved validation metrics. Sources: PyTorch, TorchVision, and scikit-learn documentation.

Representative prompt example

> "Implement a plain CNN baseline and a ResNet-50 classifier with an option to freeze the backbone, both returning 30 logits, and a checkpoint helper that keeps the epoch with the best validation Macro-F1 and the earlier epoch on ties."

4.4 Final Results Analysis, Assignment Page, and Report

Tool / model	Claude (Anthropic), Claude Opus 5.5, VS Code extension
Member / verifier	Hoang Kim Cuong (2352145)
Stage	M3 Final preparation

- **Purpose and scope:** Rewrite of the Methodology, Experimental Setup, Results, Comparison and Discussion, Error Analysis, and Limitations sections from the three final runs; export of the final figures and comparison table; grouping of test errors by dish family and a visual review of 30 sampled cross-family errors; drafting of this report and `report/report.md` from the saved artifacts. Files: `assignment2.html`, `assignment2.tex`, `results/final_comparison.csv`, `results/figures/final_*`, `report/report.md`, and `AI_USAGE.md`.

- **Edits:** Requested shorter, objective text with highlighted key points; corrected label word order; removed sections that duplicated other deliverables.
- **Verification:** Every reported number was traced to the run artifacts (`summary.json`, `evaluation_*.json`, `inference_benchmark.json`, `history.json`, `per_class_test.csv`, `predictions_test.csv`) in `results/kaggle/`; figures and the CSV were produced with `experiments.plot_curves` and `experiments.compare`. Sources: the Course Project Handbook (Sections 3 and 20–21).

Representative prompt example

> "Make the Assignment 2 methodology more concise, write it objectively without personal pronouns, and make sure no content is repeated."

4.5 Results Provenance and Responsibility

- **Results provenance:** Final metrics come from the saved `summary.json`, `evaluation_val.json`, `evaluation_test.json`, and `inference_benchmark.json` files of the three final runs. Curves, confusion matrices, and prediction examples visualize outputs of executed training and evaluation runs, not AI-estimated results.
- **Final responsibility:** Tran Manh Thang is responsible for the scaffold, environment, and experiment runner; Phan Tan Loc for the data audit, split, and EDA; and Hoang Kim Cuong for the models, training, evaluation, and final results analysis. Each member verified the AI-assisted work in the assigned area, and Group 09 retains responsibility for the submitted code, results, and interpretations.

5 Reproducibility

The final submission, including the report source and compiled PDF, is identified by the tag `a2-m3-final-submission`.

The source repository provides the implementation and configurations. Saved checkpoints are available in the M3 final submission release, in `a2-m3-final-runs-20261103.zip`.

Model	Configuration	Recorded final run ID
Scratch CNN	`baseline_cnn`	`20261103T145419Z_7a003524`
Frozen ResNet-50	`resnet50_frozen`	`20261103T160427Z_d37a574f`
Full ResNet-50	`resnet50_finetune`	`20261103T170311Z_ce0f6099`

Table 18: Saved runs underlying the reported final results.

Each run folder in `results/kaggle/` links the best checkpoint, `summary.json`, `evaluation_val.json`, `evaluation_test.json`, `inference_benchmark.json`, `history.json`, and figures to its resolved configuration (`config.json`), manifest hashes, environment record (`pip_freeze.txt`, `gpu.txt`), and source snapshot (`source_provenance.json`, `source_snapshot.zip`). Installation, dataset preparation, training, and evaluation instructions are provided in the Assignment 2 README, and Kaggle training steps in `docs/kaggle.md`. Use the released code and matching configuration when evaluating a saved checkpoint: evaluation reads the configuration on disk. Because training used cuDNN benchmark mode and mixed precision on a Kaggle T4, reruns are expected to match closely but not bit-for-bit.

References

[1] Instructor's lecture slides. `08-MLP.pdf`, PDF p. 7.

[2] Instructor's lecture slides. `09-TrainingANN.pdf`, PDF p. 11.

[3] Instructor's lecture slides. `06-Metrics.pdf`, PDF pp. 13, 19.

[4] Instructor's lecture slides. `01-Conv2d-Overview.pdf`, PDF p. 27.

[5] Instructor's lecture slides. `05-Normalization.pdf`, PDF p. 11.

[6] Instructor's lecture slides. `08-CNN-Architectures.pdf`, PDF pp. 17–19, 21, 50, 51.

[7] Instructor's lecture slides. `01-Conv2d-Overview.pdf`, PDF pp. 30–31.

[8] Instructor's lecture slides. `05-Normalization.pdf`, PDF pp. 12, 31.

[9] Instructor's lecture slides. `08-MLP.pdf`, PDF p. 19.

[10] Instructor's lecture slides. `07-CNN-Building-Blocks.pdf`, PDF pp. 6, 15.

[11] Instructor's lecture slides. `09-Augmentation.pdf`, PDF pp. 8, 26, 36, 39.

[12] Instructor's lecture slides. `04-Practical-Finetuning-and-Efficiency.pdf`, PDF p. 7.

[13] K. He, X. Zhang, S. Ren, and J. Sun. Deep Residual Learning for Image Recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.

[14] Quan Dang. *30VNFoods: Vietnamese Foods Image Dataset*, version 11. Kaggle. `https://www.kaggle.com/datasets/quandang/vietnamese-foods`.